Complete Guide to Digital Filters & Windows

FIR, IIR, Adaptive, Classical, and Modern Filter Design

Plus: Complete Window Functions Reference

Part I — Digital Filters Overview

A **digital filter** is a system that processes discrete-time signals to enhance desired components or suppress unwanted ones. The two fundamental architectures are:

Type	Structure	Memory
FIR (Finite Impulse Response)	Non-recursive (feed-forward only)	Finite — output depends on current + past N inputs
IIR (Infinite Impulse Response)	Recursive (includes feedback)	Infinite — output depends on inputs AND past outputs

Why Two Types?

FIR filters are always stable and can have exactly linear phase (no phase distortion). IIR filters are more computationally efficient (achieve sharper cutoffs with fewer coefficients) but can be unstable and always have nonlinear phase.

1. FIR Filters — Finite Impulse Response

1.1 Mathematical Foundation

The difference equation of an FIR filter of order N:

$$y[n] = \Sigma\ (k{=}0 \text{ to } N)\ \ b[k] \cdot x[n-k]$$

where **b[k]** are the filter coefficients (the impulse response), and **x[n−k]** are delayed inputs. This is a convolution of the input with the impulse response.

1.2 Z-Transform & Transfer Function

$$H(z) = b[0] + b[1]\cdot z^{-1} + b[2]\cdot z^{-2} + \ldots + b[N]\cdot z^{-N}$$

All poles are at the origin (z = 0), so FIR filters are **inherently stable**. Zeros can be placed anywhere to shape the frequency response.

1.3 Linear Phase Property

If the coefficients are **symmetric** (b[k] = b[N−k]) or **antisymmetric** (b[k] = −b[N−k]), the filter has exactly linear phase:

$$\varphi(\omega) = -\omega \cdot N/2 + \text{constant}$$

This means all frequencies are delayed by the same amount — **no phase distortion**. Critical for audio, video, and communications.

1.4 FIR Design Methods

A. Window Method

Start with the ideal (infinite) impulse response h_ideal[n], then multiply by a window w[n] to truncate it:

$$h[n] = h_ideal[n] \cdot w[n]$$

Trade-off: longer window → sharper cutoff but more coefficients.

B. Frequency Sampling Method

Specify the desired frequency response H[k] at uniformly spaced frequencies, then use inverse DFT to get h[n]. Fast but ripples in passband/stopband.

C. Parks-McClellan (Equiripple) Algorithm

Uses Remez exchange to find the **optimal** FIR filter (minimizes maximum error in passband and stopband). MATLAB: `firpm()`. This is the gold standard for FIR design.

D. Least-Squares Method

Minimizes the mean-squared error between H_desired(ω) and H_actual(ω). MATLAB: `firls()`.

1.5 FIR Filter Types

Type	Symmetry	Use Case
Type I	Even length, symmetric	Lowpass, highpass, bandpass
Type II	Odd length, symmetric	Lowpass, bandpass (NOT highpass)
Type III	Even length, antisymmetric	Differentiators, Hilbert transformers
Type IV	Odd length, antisymmetric	Differentiators, 90° phase shifters

Practical Note

For your speaker recognition project, you won't design FIR filters manually — the pre-emphasis filter is IIR. But FIR filters are used in many DSP applications (equalizers, matched filters, pulse shaping).

2. IIR Filters — Infinite Impulse Response

2.1 Mathematical Foundation

The general IIR difference equation:

$$y[n] = \Sigma(k=0 \text{ to } M)\ b[k]\cdot x[n-k] - \Sigma(k=1 \text{ to } N)\ a[k]\cdot y[n-k]$$

The first sum is the **feedforward** part (zeros), the second is the **feedback** part (poles). This is recursive — the output depends on past outputs.

2.2 Z-Transform & Transfer Function

$$H(z) = [b[0] + b[1]\cdot z^{-1} + ... + b[M]\cdot z^{-M}] / [1 + a[1]\cdot z^{-1} + ... + a[N]\cdot z^{-N}]$$

Poles can be anywhere. **Stability condition**: all poles must lie <u>inside</u> the unit circle (|z| < 1).

2.3 Why IIR? — Efficiency

IIR filters achieve much sharper cutoffs with fewer coefficients than FIR. Compare:

- • FIR lowpass (100 Hz cutoff, fs=1000 Hz, 60 dB stopband): ~150 coefficients
- • IIR lowpass (same specs): ~8 coefficients

The trade-off: IIR has nonlinear phase (phase distortion).

2.4 Classical IIR Filter Families

All IIR filters are designed first as **analog prototypes** (s-domain), then converted to digital (z-domain) via bilinear transform or impulse invariance. The four classical families:

2.4.1 Butterworth Filter

Characteristic: Maximally flat passband (no ripple)

Magnitude-squared frequency response:

$$|H(j\omega)|^2 = 1 / [1 + (\omega/\omega c)^{(2N)}]$$

where N = filter order, ωc = cutoff frequency (−3 dB point).

Property	Value
Passband	Monotonic, maximally flat at DC
Stopband	Monotonic roll-off (−20N dB/decade)
Phase	Moderately nonlinear
Pole pattern	Equally spaced on semicircle in s-plane
Use case	General-purpose, audio (smooth response)

MATLAB: `[b,a] = butter(N, Wn);`

Where: N = order, Wn = normalized cutoff (0 to 1, where 1 = Nyquist)

Example

6th-order Butterworth lowpass at 500 Hz with fs=2000 Hz: [b,a] = butter(6, 500/1000); freqz(b, a, 512, 2000); % plot response

2.4.2 Chebyshev Type I Filter

Characteristic: Equiripple passband, monotonic stopband

$$|H(j\omega)|^2 = 1 / [1 + \varepsilon^2 \cdot T_n^2(\omega/\omega c)]$$

where $T_n(x)$ = Chebyshev polynomial of order N, ε controls passband ripple.

Property	Value
Passband	Equiripple (oscillates between 1 and $1-\varepsilon^2$)
Stopband	Monotonic (steeper than Butterworth)
Phase	More nonlinear than Butterworth
Pole pattern	Lie on ellipse in s-plane
Use case	When sharper cutoff needed, can tolerate passband ripple

MATLAB: `[b,a] = cheby1(N, Rp, Wn);`

Where: Rp = passband ripple in dB (e.g., 0.5 dB)

When to Use

Choose Chebyshev Type I when you need a sharper roll-off than Butterworth and can accept small passband oscillations (e.g., anti-aliasing filters in ADCs).

2.4.3 Chebyshev Type II (Inverse Chebyshev)

Characteristic: Monotonic passband, equiripple stopband

This is the inverse of Type I — ripple moved from passband to stopband.

Property	Value
Passband	Monotonic, flat (no ripple)
Stopband	Equiripple (oscillates at level −Rs dB)
Phase	Similar to Type I
Pole pattern	Poles AND zeros (Type I has only poles)
Use case	When flat passband is critical but sharp stopband is needed

MATLAB: `[b,a] = cheby2(N, Rs, Wn);`

Where: Rs = stopband attenuation in dB (e.g., 40 dB)

> **Practical Difference**
>
> Type I affects the passband (where your signal lives). Type II affects the stopband (where noise/interference lives). Choose based on what you can tolerate.

2.4.4 Elliptic (Cauer) Filter

Characteristic: Equiripple in BOTH passband and stopband

The most aggressive IIR design — achieves the **sharpest possible transition** for a given order.

Property	Value
Passband	Equiripple (±Rp dB)
Stopband	Equiripple (−Rs dB)
Transition width	Narrowest of all classical filters
Phase	Most nonlinear (worst group delay variation)
Use case	When minimal filter order is critical (embedded systems)

MATLAB: `[b,a] = ellip(N, Rp, Rs, Wn);`

Where: Rp = passband ripple, Rs = stopband attenuation

> **Trade-Off Summary**

Elliptic gives you the sharpest cutoff but the most phase distortion. If linear phase matters (audio, video), avoid elliptic. If you just need to kill frequencies fast (anti-aliasing, decimation), elliptic is perfect.

2.5 IIR Filter Comparison — All Four Families

For a lowpass filter with the same passband/stopband specs, here's how they compare:

Filter Type	Order (N)	Passband	Stopband	Phase	Best For
Butterworth	Highest	Flat	Monotonic	Moderate	General-purpose, audio
Chebyshev I	Medium	Ripple	Monotonic	Nonlinear	Sharp cutoff, tolerate passband ripple
Chebyshev II	Medium	Flat	Ripple	Nonlinear	Flat passband required
Elliptic	Lowest	Ripple	Ripple	Most nonlinear	Minimal order, phase not critical

Design Decision Tree

Need linear phase? → Use FIR Need sharp cutoff, can't use many coefficients? → IIR └─ Can tolerate passband ripple? → Chebyshev I or Elliptic └─ Need flat passband? → Butterworth or Chebyshev II └─ Absolute minimal order? → Elliptic

2.6 Bilinear Transform

Converts analog filter H(s) to digital H(z):

$$s \leftarrow (2/T) \cdot [(z - 1) / (z + 1)]$$

where T = sampling period. This maps the entire jω-axis (analog) onto the unit circle (digital) with **no aliasing**, but causes **frequency warping**. Pre-warping corrects this:

$$\omega d = (2/T) \cdot \tan(\omega a \cdot T/2)$$

3. Adaptive Filters

Adaptive filters learn their coefficients in real-time by minimizing an error signal. Used when the signal or noise characteristics are unknown or time-varying.

3.1 Structure

An adaptive filter has:

- • Input signal x[n]
- • Desired signal d[n] (reference)
- • Filter output y[n] = Σ w[k]·x[n−k]
- • Error e[n] = d[n] − y[n]
- • Adaptation algorithm that updates weights w[k] to minimize e[n]

3.2 LMS — Least Mean Squares

The Algorithm:

$$w[n+1] = w[n] + \mu \cdot e[n] \cdot x[n]$$

where μ = step size (learning rate), e[n] = error, x[n] = input vector.

Property	Value
Complexity	O(N) per iteration (very low)
Convergence	Slow but guaranteed if 0 < μ < 2/λmax
Tracking	Good for slowly varying systems
Noise sensitivity	Sensitive (uses instantaneous gradient)
Use case	Echo cancellation, noise cancellation, equalization

Choosing μ

Small μ → slow convergence, low steady-state error Large μ → fast convergence, high steady-state error (can diverge) Typical range: 0.01 to 0.1

3.3 RLS — Recursive Least Squares

The Algorithm (simplified):

$$w[n] = w[n-1] + K[n] \cdot e[n]$$

where K[n] is the Kalman gain vector, computed from the autocorrelation matrix of the input.

Property	Value
Complexity	$O(N^2)$ per iteration (high)
Convergence	Fast (typically N iterations)
Tracking	Excellent for rapidly varying systems
Noise sensitivity	Low (uses all past data)
Use case	Channel equalization, system identification

LMS vs RLS

LMS: Cheap, slow, simple — use when computation is limited RLS: Expensive, fast, complex — use when fast tracking is critical RLS converges in ~N samples, LMS may take 10N to 100N samples

3.4 Applications of Adaptive Filters

- **1. Acoustic Echo Cancellation (AEC)**
  - Remove speaker output from microphone in teleconferencing
- **2. Noise Cancellation**
  - Active noise cancellation headphones use LMS to predict and cancel ambient noise
- **3. Channel Equalization**
  - Compensate for distortion in communication channels (wireless, fiber optics)
- **4. Beamforming**
  - Microphone arrays use adaptive filters to focus on one speaker

4. Kalman Filter

The Kalman filter is NOT a frequency-domain filter — it's a **state-space estimator** that optimally estimates the state of a dynamic system from noisy measurements.

4.1 The Problem It Solves

You have:

- • A system with hidden state x[n] (e.g., position, velocity of a car)
- • Noisy measurements z[n] (e.g., GPS readings)
- • A model of how the state evolves: x[n+1] = F·x[n] + w[n]
- • A model of measurements: z[n] = H·x[n] + v[n]

Goal: Estimate x[n] optimally given all past measurements.

4.2 The Kalman Filter Algorithm

Prediction step:

$$\hat{x}[n|n-1] = F \cdot \hat{x}[n-1|n-1]$$

$$P[n|n-1] = F \cdot P[n-1|n-1] \cdot F^T + Q$$

Update step (when measurement z[n] arrives):

$$K[n] = P[n|n-1] \cdot H^T \cdot (H \cdot P[n|n-1] \cdot H^T + R)^{-1}$$

$$\hat{x}[n|n] = \hat{x}[n|n-1] + K[n] \cdot (z[n] - H \cdot \hat{x}[n|n-1])$$

$$P[n|n] = (I - K[n] \cdot H) \cdot P[n|n-1]$$

where:

- • x̂ = state estimate
- • P = error covariance matrix
- • K = Kalman gain
- • Q = process noise covariance
- • R = measurement noise covariance

4.3 Why It's Optimal

If the system is linear and the noise is Gaussian, the Kalman filter is the **minimum mean-squared error (MMSE)** estimator. No other estimator can do better (Wiener filter is a special case).

4.4 Applications

Domain	Application
Navigation	GPS tracking, inertial navigation (sensor fusion)
Robotics	SLAM (Simultaneous Localization and Mapping)
Economics	Stock price prediction, econometric modeling
Signal Processing	Denoising, tracking radar targets
Control	Optimal state feedback control

Kalman vs Adaptive Filters

Adaptive filters (LMS/RLS) work in signal space (filter coefficients adapt). Kalman filter works in state space (estimates hidden states). Both are recursive and optimal in their respective domains.

4.5 Extended Kalman Filter (EKF)

For **nonlinear** systems, the standard Kalman filter doesn't work. The EKF linearizes the system around the current estimate using the Jacobian. Used in GPS, robotics, and control.

5. Median Filter

The median filter is a **nonlinear** filter used primarily for impulse noise removal (salt-and-pepper noise in images, spikes in 1D signals).

5.1 How It Works

For each sample, replace it with the **median** of its neighborhood:

$$y[n] = \text{median}\{ x[n-k], ..., x[n], ..., x[n+k] \}$$

where the window size is 2k+1 (usually odd).

5.2 Properties

Property	Value
Linearity	Nonlinear (median is not a linear operation)
Impulse response	None (no transfer function, not an LTI system)
Edge preservation	Excellent (doesn't blur edges like linear filters)
Noise type	Best for impulse noise (isolated spikes)
Computational cost	O(N log N) per sample (sorting)

5.3 Why Use Median Filter?

Linear filters (moving average, Gaussian) blur edges when removing noise. The median filter preserves edges because it picks an existing value — it doesn't average.

Example — Salt-and-Pepper Noise

Image with random white/black pixels: • Gaussian filter → blurs entire image • Median filter → removes spikes, keeps edges sharp MATLAB: filtered = medfilt1(signal, window_size);

5.4 Variants

- **Weighted Median Filter:** Weights closer samples more heavily
- **Adaptive Median Filter:** Changes window size based on local noise level
- **Recursive Median Filter:** Uses previously filtered outputs in the window

6. Additional Modern Filters

6.1 Wiener Filter

Purpose: Optimal linear filter for signal estimation in the presence of noise

The Wiener filter minimizes mean-squared error between the filter output and the desired signal. Requires knowledge of signal and noise power spectra.

$$H(\omega) = S_{xx}(\omega) / [S_{xx}(\omega) + S_{nn}(\omega)]$$

where S_{xx} = signal power spectrum, S_{nn} = noise power spectrum.

6.2 Matched Filter

Purpose: Maximize SNR for detecting a known signal in noise

The impulse response is the **time-reversed** version of the signal you're looking for:

$$h[n] = s[-n]$$

Used in radar, sonar, digital communications (correlation receivers).

6.3 Notch Filter

Purpose: Remove a very narrow frequency band (e.g., 60 Hz power line hum)

Typically implemented as an IIR filter with a pair of complex conjugate zeros on the unit circle at the unwanted frequency. MATLAB: `iirnotch(w0, BW)`

6.4 Comb Filter

Purpose: Periodic notches/peaks in frequency response

$$H(z) = (1 \pm z^{-N}) / 2$$

Used in audio (flanging, echo effects), video (artifact removal).

6.5 Savitzky-Golay Filter

Purpose: Smooth data while preserving peaks and derivatives

Fits a polynomial to a sliding window, then evaluates the polynomial at the center point. Widely used in spectroscopy, chromatography. MATLAB: `sgolayfilt()`

6.6 Hilbert Transform Filter

Purpose: 90° phase shift — computes the analytic signal

Used to extract instantaneous amplitude/frequency/phase. MATLAB: `hilbert()`

Part II — Window Functions

Window functions taper the edges of a finite signal segment to reduce spectral leakage when applying the DFT/FFT.

General Form

All windows can be written as:

$$w[n] = a_0 - a_1 \cdot \cos(2\pi n/N) + a_2 \cdot \cos(4\pi n/N) - \ldots$$

Different windows are just different choices of coefficients a_0, a_1, a_2, ...

Key Performance Metrics

Metric	Meaning
Main lobe width	Frequency resolution (wider → worse resolution)
Side lobe level	Spectral leakage (higher → more leakage)
Side lobe fall-off	How fast side lobes decay with frequency
Scalloping loss	Loss when signal is between DFT bins
Equivalent noise BW	Effective bandwidth (affects SNR)

W1. Rectangular Window

$$w[n] = 1, \quad 0 \le n \le N-1$$

(No windowing — equivalent to multiplying by 1)

Property	Value
Main lobe width	Narrowest (best frequency resolution)
First side lobe level	−13 dB (very high — worst leakage)
Side lobe fall-off	−6 dB/octave (slow)
Use case	When signal fits exactly into DFT frame (no truncation)

W2. Hamming Window

$$w[n] = 0.54 - 0.46 \cdot \cos(2\pi n/(N-1))$$

Property	Value
Main lobe width	1.3× wider than rectangular
First side lobe level	−43 dB (excellent suppression)
Side lobe fall-off	−6 dB/octave
Use case	General-purpose (speech, audio, vibration analysis)

Why 0.54 and 0.46?

These constants place a zero of the window's DTFT exactly on the first side lobe of the rectangular window's DTFT, canceling it. This gives the −43 dB suppression.

W3. Hann (Hanning) Window

$$w[n] = 0.5 - 0.5 \cdot \cos(2\pi n/(N-1))$$

(Also called raised cosine window)

Property	Value
Main lobe width	1.5× wider than rectangular
First side lobe level	−32 dB
Side lobe fall-off	−18 dB/octave (faster than Hamming)

Use case	When rapid side lobe decay is needed

W4. Blackman Window

$$w[n] = 0.42 - 0.5 \cdot \cos(2\pi n/(N-1)) + 0.08 \cdot \cos(4\pi n/(N-1))$$

Property	Value
Main lobe width	1.7× wider than rectangular
First side lobe level	−58 dB (very good)
Side lobe fall-off	−18 dB/octave
Use case	High dynamic range applications (audio mastering)

W5. Blackman-Harris Window

A 4-term generalization of Blackman:

$$w[n] = a_0 - a_1 \cdot \cos(2\pi n/N) + a_2 \cdot \cos(4\pi n/N) - a_3 \cdot \cos(6\pi n/N)$$

Standard coefficients: a_0=0.35875, a_1=0.48829, a_2=0.14128, a_3=0.01168

Property	Value
Main lobe width	2.0× wider than rectangular
First side lobe level	−92 dB (excellent)
Side lobe fall-off	−6 dB/octave
Use case	Spectral analysis requiring very low leakage

W6. Kaiser Window

A **parametric** window — you control the trade-off:

$$w[n] = I_0(\beta \cdot \sqrt{(1-[(n-N/2)/(N/2)]^2)}) \;/\; I_0(\beta)$$

where I_0 = modified Bessel function of the first kind, β = shape parameter.

β value	Side lobe level	Main lobe width
0	−13 dB	Narrowest
3	−40 dB	Medium
6	−60 dB	Wider
9	−80 dB	Widest

MATLAB

w = kaiser(N, beta); Choose beta based on desired stopband attenuation: beta ≈ 0.1102·(A − 8.7) where A = attenuation in dB Example: For A = 60 dB → beta ≈ 5.65

Window Comparison Summary

Window	Main Lobe	Side Lobe	Fall-off	Best For
Rectangular	Narrowest	−13 dB	−6 dB/oct	No windowing needed (perfect periodicity)
Hamming	1.3×	−43 dB	−6 dB/oct	General-purpose (speech, audio)
Hann	1.5×	−32 dB	−18 dB/oct	Faster side lobe decay
Blackman	1.7×	−58 dB	−18 dB/oct	High dynamic range
Blackman-Harris	2.0×	−92 dB	−6 dB/oct	Very high dynamic range
Kaiser (β=6)	1.8×	−60 dB	Adjustable	Tunable trade-off

Choosing a Window — Decision Tree

Need best frequency resolution? → Rectangular (but only if no truncation) General-purpose DSP? → Hamming (speech, vibration, default choice) Audio analysis with strong harmonics? → Blackman or Blackman-Harris Need to tune the trade-off? → Kaiser (adjust β to taste) Rapid side lobe decay? → Hann

Additional Advanced Windows

W7. Flat-Top Window

Designed to have a very flat passband (accurate amplitude measurement).

Trade-off: Very wide main lobe (poor frequency resolution). Used in FFT analyzers for calibration.

W8. Tukey (Tapered Cosine) Window

Rectangular in the middle, cosine taper at edges. Parameter α controls taper width.

α=0 → rectangular, α=1 → Hann. MATLAB: `tukeywin(N, alpha)`

W9. Gaussian Window

$$w[n] = \exp(-\tfrac{1}{2}[(n-(N-1)/2) / (\sigma\cdot(N-1)/2)]^2)$$

Parameter σ controls width. Used in time-frequency analysis (Gabor transform).

W10. Dolph-Chebyshev Window

Designed to have the **narrowest main lobe** for a given side lobe level. Optimal in that sense. MATLAB: `chebwin(N, attenuation)`

W11. Nuttall Window

4-term cosine window (like Blackman-Harris but different coefficients). Slightly better side lobe suppression. MATLAB: `nuttallwin(N)`

Part III — Practical Filter Design Workflow

Step 1: Define Requirements

- • Filter type: lowpass, highpass, bandpass, bandstop
- • Passband: frequency range to preserve
- • Stopband: frequency range to reject
- • Transition width: steepness requirement
- • Ripple/attenuation specs

Step 2: Choose Architecture

Decision

Need linear phase (audio, video, communications)? → FIR Sharper cutoff, fewer coefficients, phase not critical? → IIR

Step 3: Choose Family (if IIR)

- • Flat passband + moderate stopband → **Butterworth**
- • Sharp cutoff + can tolerate passband ripple → **Chebyshev I**
- • Flat passband + equiripple stopband → **Chebyshev II**
- • Sharpest possible + minimal order → **Elliptic**

Step 4: Design in MATLAB

```
% FIR design (Parks-McClellan)
b = firpm(N, [0 fp fs 1], [1 1 0 0]);  % equiripple
% IIR design (Butterworth lowpass)
[b,a] = butter(N, Wn);
% View frequency response
freqz(b, a, 512, fs);
```

Step 5: Verify & Test

- • Check frequency response (magnitude, phase)
- • Check group delay (linear phase?)
- • Test on real signals
- • Verify stability (all poles inside unit circle for IIR)

Summary & Quick Reference

Filter Types at a Glance

FIR: Linear phase, always stable, more coefficients IIR: Efficient, sharp cutoff, nonlinear phase, can be unstable Adaptive: Learn coefficients on-the-fly (LMS, RLS) Kalman: State estimation, not frequency filtering Median: Nonlinear, preserves edges, removes impulse noise

IIR Families at a Glance

Butterworth: Flat passband, general-purpose Chebyshev I: Sharp cutoff, passband ripple OK Chebyshev II: Flat passband, stopband ripple OK Elliptic: Sharpest, both ripples, worst phase

Windows at a Glance

Hamming: Default choice (−43 dB, good balance) Hann: Fast side lobe decay (−32 dB, −18 dB/oct) Blackman: High dynamic range (−58 dB) Kaiser: Adjustable trade-off (tune β for your needs) Rectangular: Only when no windowing needed

This guide covers the complete landscape of digital filters and windows used in DSP. Use it as a reference when designing systems, reading papers, or implementing algorithms. Each filter family has its place — the key is knowing when to use which one.